# The listening telephone — automating speech recognition over the PSTN

**K J Power**

*Recognition of human speech remains a very difficult task for machines, although one that humans effortlessly achieve every day. Nevertheless, the underlying technology has matured enough for many advanced speech-based applications to become commercially viable.*

*This paper gives an overview of the basic acoustic pattern matching elements used for state-of-the-art recognition, highlighting some of the problems that arise in building real working systems and the solutions that have been developed to address them.*

## 1. Introduction

The impression often evoked by the term 'speech recognition' is of effortless spontaneous conversation with a computer. This ideal has long been inspired by examples from science fiction like HAL from '2001: A Space Odyssey', or the ever-alert shipboard computer on the *Enterprise*. These ideals embody the goals of today's speech researchers, but while impressive advances have been made in recent years achieving them is still some way off, requiring the ability not only to recognise speech but also to understand it. Nevertheless, with today's technology, if systems are designed carefully and the recognition domain is suitably restricted, tasks ranging from dictation of letters to catalogue shopping by telephone can be automated.

What makes automatic speech and speaker recognition (ASR)[1] such fascinating challenges is the variability seen (or rather heard) in the signals that must be classified. First, speech signals exhibit enormous variability even for a speaker repeating the same sound under identical conditions; add to this different background noises, ranging from interfering speech to traffic noise, and compound the issue by passing the signal through a telephone handset of unknown type. Finally, send it through the telephone network with its own noise and restricted bandwidth (300—3500 Hz) and the scale of the problem can be appreciated.

Even more variability arises if the speech originates from more than one speaker, because of differences in vocal tract, accent and speaking style. Also, the level and type of background noise have significant effects on the speech signal.

Some systems remove a lot of this variability by restricting themselves to working for a single speaker. Such **speaker-dependent** recognisers are trained on the speech from a single user and are usually only used over a single channel. These systems require an enrolment session where the user provides examples of their speech. A typical application of speaker-dependent recognition is PC-based dictation. The task of speaker recognition is, of course, necessarily speaker dependent. **Speaker-independent** recognisers, on the other hand, are designed to accept speech from any speaker and have the advantage that they do not require any enrolment from new users. They are therefore the only possibility for many telephony-based services where callers are unknown in advance.

There are many types of recogniser — for both speech and speaker recognition — each optimised for a different task. The mechanism for encapsulating task constraints (those limitations in the scope of the application domain that make recognition feasible) is described for several real systems and practical issues are outlined.

## 2. Basic components

The basic components of both speech and speaker recognisers are very similar. These are illustrated in Fig 1.

### *2.1 Speech capture*

The input to a recogniser is the physical speech signal. During speech capture the signal is sampled, digitally encoded and stored. In many telephony-based applications, detection of when to start and end speech capture is also necessary.

[1] The abbreviation ASR refers to automatic speech and speaker recognition throughout this paper.

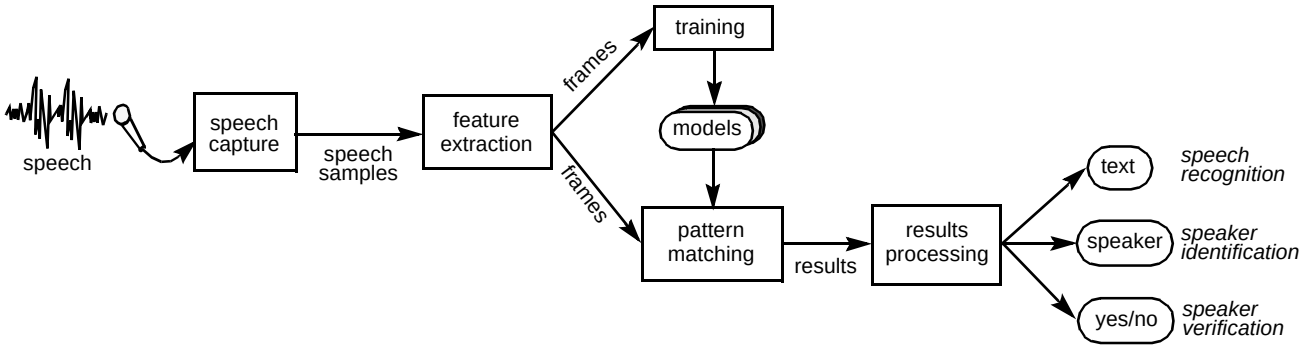



Fig 1 Common components of speech and speaker recognisers.

### *2.2 Feature extraction*

Recognition is not performed directly on the sampled signal because it contains a great deal of redundant information. Instead, the signal is analysed to extract key features for either speech intelligibility or speaker characterisation. This process differs slightly for speech and speaker recognition, but in either case the output is a sequence of speech **feature vectors** or **frames**.

### *2.3 Modelling speech sounds*

Both speech and speaker recognisers attempt to identify patterns in the feature vectors corresponding to speech units such as words, phrases or phonemes. This is done using one or more models of each required speech unit. A statistical model — known as the hidden Markov model (HMM) — is most widely used for recognition. Models must first be created in a separate training phase before being used for recognition.

### *2.4 Pattern matching*

Speech signals are usually assumed to be a sequence of speech sounds. Pattern matching is the process of finding the best model sequence that matches the signal. This is usually done by a parser. The output is the recognition results which typically includes a set of hypothesised model sequences with accompanying scores.

### *2.5 Results processing*

Further analysis is performed on the pattern matching results to arrive at the recognition decision. The process is different for speech and speaker recognisers to produce the desired form of output as shown in Fig 1.

## 3. Speech capture

Although the process of sampling the speech signal is relatively straightforward, accurate detection of when to stop speech capture is by no means easy. Similarly, in some systems a means of detecting when speech has started is needed. Both these scenarios are an integral part of the complete speech capture process often required for a live system.

### *3.1 End-of-speech detection*

For many applications the recognition component does not run continuously. Instead, a recognition process is started and must stop at some point, returning control to the application. One option is to stop recognition after a fixed time, but this introduces a delay if the user finishes speaking before the recognition window closes. Also, if the user delays before speaking, there is a risk of prematurely stopping speech capture.

A better strategy is to analyse the pattern-matching results as shown in Fig 2. This requires models of non-speech sounds, i.e. background noise or silence, so that the entire speech signal can be appropriately matched. The results contain a hypothesised model sequence. A sequence ending in one or more noise models indicates that the corresponding final portion of the signal is non-speech. Once post-speech noise — along with various other criteria — is detected, speech capture can be halted.

### *3.2 Overriding a spoken prompt*

In a spoken-dialogue application, users become familiar with the system and it is desirable that they can override prompts and so speed up their transactions. This is known as cut-through or ‘barge in’.

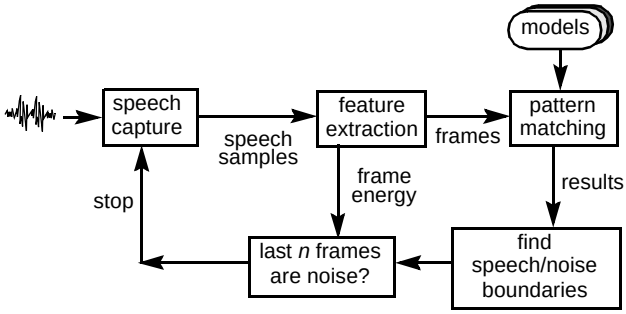



Fig 2 End-of-speech detection.

Unfortunately, incoming speech becomes corrupted with echo from the outgoing prompt with detrimental effects on recognition accuracy. In a telephony application for example this is further exacerbated by both acoustic and electrical coupling of the incoming and outgoing signal inherent in the telephone connection. This is illustrated in Fig 3.

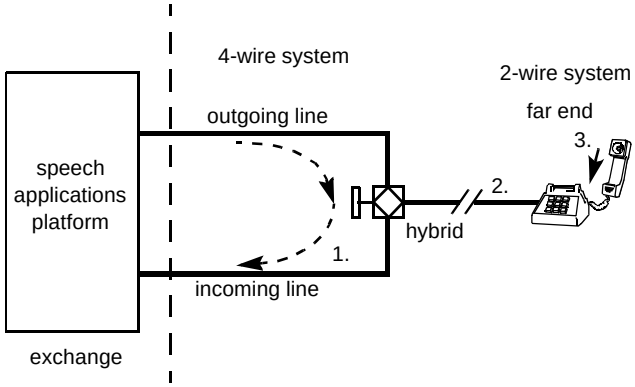



Fig 3 Causes of echo in the PSTN. (1. impedance mismatch in hybrid; 2. electro-acoustic coupling; 3. acoustic feedback in the handset).

One effective solution is to detect an override as soon as possible and deactivate the prompt. If the system plays out a short (non-overrideable) welcome announcement this may be used to estimate the echo level. Detection of echo is done on a per-frame basis for speed. A frame can be classified as speech, echo or noise. Energy-based features allow some discrimination — as users tend to speak louder as the background noise level increases (this is known as the Lombard effect [1]).

Even when the prompt is rapidly deactivated, some echo will still be mixed with the speech signal. This can reduce recognition accuracy by up to 10% for small vocabularies. Techniques similar to those used for noise robustness can be used to estimate the echo spectrum and (assuming signals are additive in the spectral domain) subtract it from the input spectrum and hence maintain recognition accuracy.

## 4. Feature extraction

During feature extraction, the input speech samples are transformed into a domain where information of importance for recognition is preserved and redundant information discarded. This signal processing stage is usually quite computationally intensive.

To give some idea of the variability of speech signals, Fig 4 shows time-domain speech waveforms for three speakers saying the word ‘zero’. These waveforms are of similar duration but with no obvious common pattern.

Patterns in the speech signal become much more evident when the signal is transformed into the frequency domain. Figure 5 illustrates corresponding spectrograms — the vertical axis denotes frequency and the horizontal axis denotes time. The power of the frequency components is proportionally represented by darkness. The resonant frequencies of the vocal tract (formants) appear clearly as the dark bands flowing across each spectrogram.

The typical process of frequency analysis of the sampled speech signal is illustrated in Fig 6. The samples are first assembled and windowed into overlapping frames. The overlap period is usually chosen to be in the range 10 to 20 ms during which speech is assumed to be quasi-stationary. A fast Fourier transform (FFT) is used to calculate the power spectrum.

The power spectrum is then organised into frequency bands according to a series of Mel scale filters as shown in Fig 7. These filters are spaced linearly to 1 kHz and then logarithmically up to the maximum frequency. The spacing of these bands is based on measurements of the sensitivity of the human ear to changes in frequency. In this way the power spectrum can be represented by about 20 Mel scale filter outputs — considerably reducing the data rate.

The dynamic range of the power spectrum is quite large and hence the logarithm is taken of the Mel filter outputs. This accords with human perception of sound intensity which is thought to vary with the logarithm of intensity.

Finally, a discrete cosine transform (DCT) is performed on the logged Mel filter outputs. This is given as:

$$C(k) = \sum_{i=0}^{N-1} f(i)\cos\left(\frac{2\pi ik}{N} + 0.5\right) \qquad k \in [0, M] \quad \text{..... (1)}$$

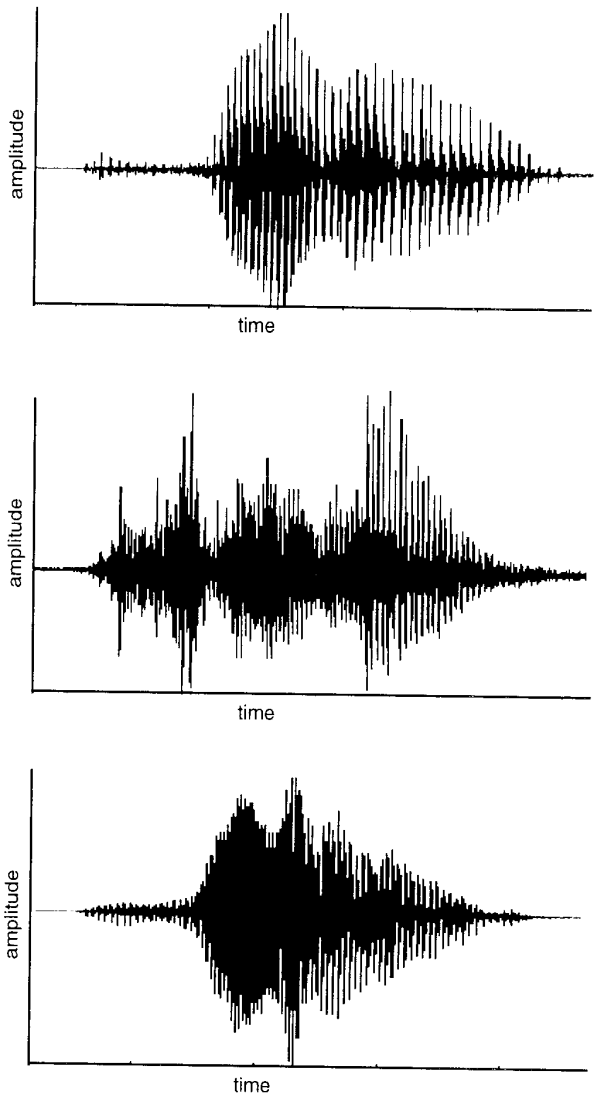



Fig 4 Time-domain waveforms for three talkers saying the word 'zero'.

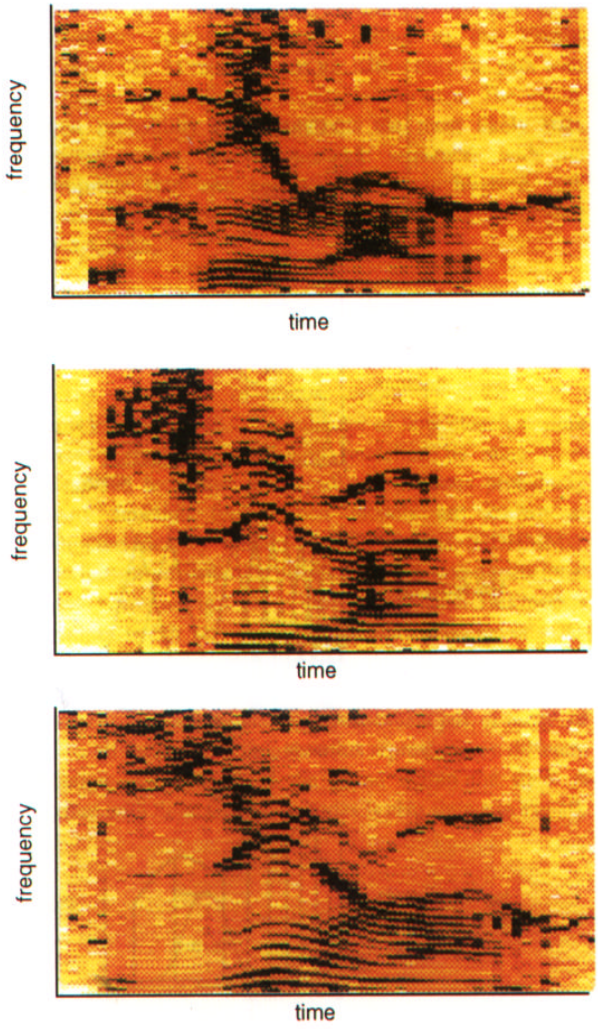



Fig 5 Spectrograms for three talkers saying word 'zero'.

where $C(k)$ is the $k$th DCT output and $f(i)$ is the $i$th of $N$ log filter bank outputs. The DCT output is known as the cepstrum (though this is not, in fact a true cepstrum which is the Fourier transform of the log of the power spectrum [2]). Two important functions are served by this transform. First, it acts as a data reduction stage. The power spectrum envelope varies slowly over the frequency range and so $M$ is usually much less than $N$ in equation (1). Secondly, the DCT outputs are relatively uncorrelated so that each output value can be assumed to be independent of every other value. This is ideal for most types of pattern matching [2], as the vector operations for training and recognition can be greatly simplified.

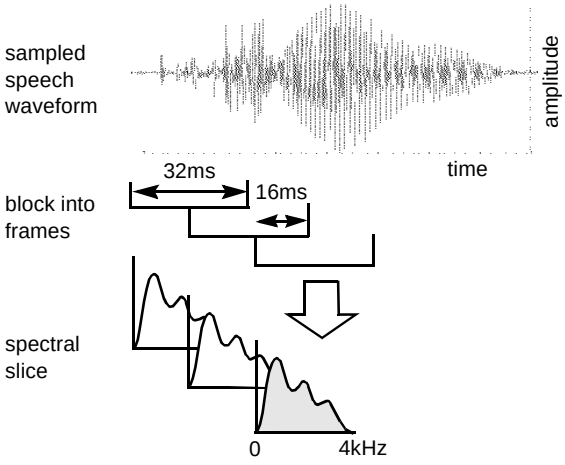



Fig 6 Spectral analysis of the speech signal.

Each feature vector contains a subset of the cepstral coefficients. In addition, the time derivative of the cepstral coefficients computed over successive, non-overlapping, frames is often included. Similarly, the differential logarithm of frame energies is also included. The final feature vector may consist of:

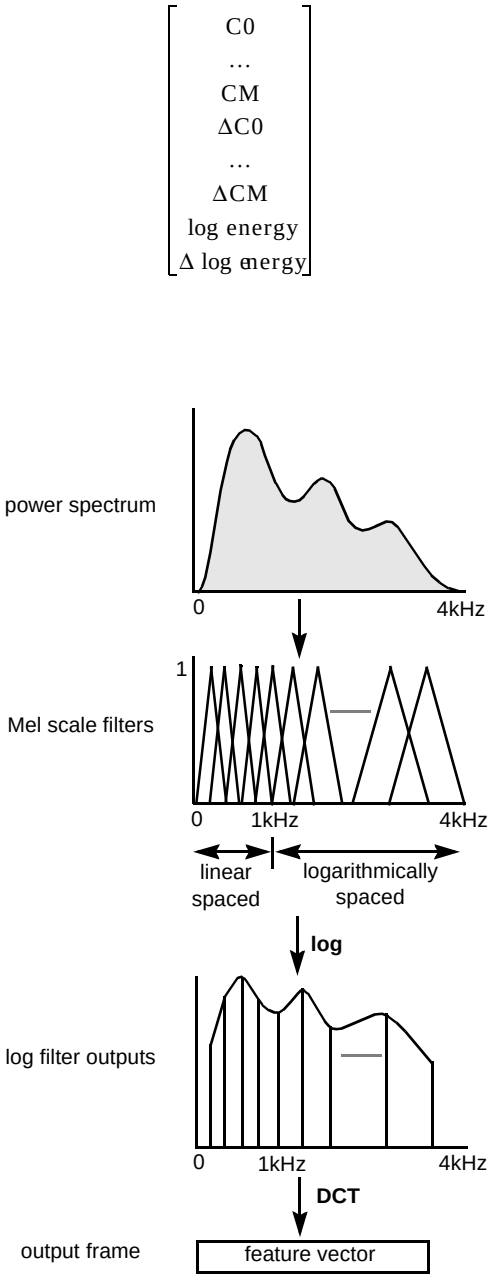



Fig 7 Mel filtering of the power spectrum.

## 5. Models — the pattern-matching elements

For recognition, some form of model of the entities to be recognised is needed. The role of a model is to represent the common patterns of similar speech sounds while also allowing for variability. This section first describes the application of statistical models known as hidden Markov models (HMM) for pattern matching. Next, the HMM training process is discussed, i.e. when the models 'learn' the sounds to be recognised. Then some of the limitations of HMMs are discussed together with details of an alternative model.

### *5.1 Hidden Markov models for recognition*

An acoustic pattern, e.g. the spoken word 'fred', is represented by a sequence of feature vectors. Different acoustic realisations of the word 'fred' give rise to different vector sequences. A good model must deal with the two following types of variability presented by vector sequences.

- Temporal variability — different vector sequences corresponding to a sound will generally differ in duration. This is due both to differences in speaking rate and the inexact nature of human speech production. Even for vector sequences of the same duration the proportion of vectors for the /*f*/ sound, say, will vary as different parts of the word will be stressed differently.
- Acoustic variability — feature vectors corresponding to an /*f*/ sound will vary within a spoken example of the word 'fred' and even more so across different speakers and channels.

HMMs can accommodate both types of variation. A signal is modelled as a **stochastic** process, i.e. can be characterised as a probabilistic function of a random variable.

An HMM consists of a number of states as illustrated in Fig 8. In this example, four states are chosen to represent each of the four sounds of the word 'fred'. Transitions between model states have an associated probability (which can be zero). Transitions are usually chosen to reflect the temporal progression of speech sounds as shown.

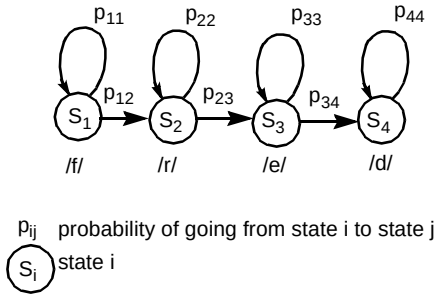


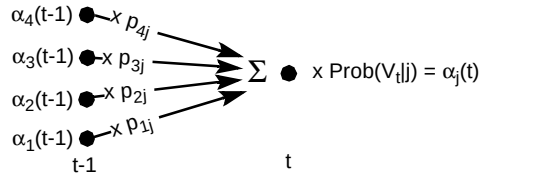

Fig 8 A hidden Markov model of the word 'fred'.

The simplest case arises when each speech frame can be uniquely assigned to one of the four states. Suppose there are six frames for an example of the word 'fred' producing the state sequence $S_1S_1S_2S_3S_3S_4$. For a Markov model the probability of a state sequence is simply the product of state transition probabilities, in this case $p_{11}p_{12}p_{23}p_{33}p_{34}$. A model output probability can be calculated for frame sequences of any length and with varying state durations.

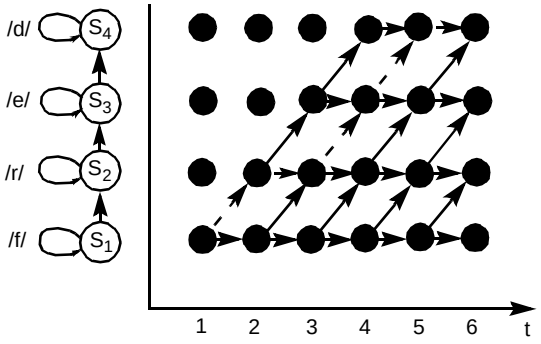
In practice, the spectral properties of a sound are too variable to allow a unique mapping of frames to states. Instead, every frame is mapped to all states with an

associated (non-zero) probability — the state output probability. As each frame cannot be uniquely matched to one state, the model output probability is the sum of all state sequence probabilities. The calculation of the state sequence probability must also incorporate the state output probability for each state in the sequence. The Markov model in this case is 'hidden' because the actual state sequence traversed through the model is unknown.

The number of possible state sequences for a frame sequence is, in practice, unfeasibly large. Assume, for simplicity that all valid state sequences start in state 1 and terminate in state 4. A word of this duration might span 30 frames of speech, giving rise to $9 \times 4^{25}$ potential state sequences. A recursive algorithm is used to efficiently traverse all possible state sequences 'on the fly'. A recursive term $\alpha j(t)$ is computed for each state $j$ for each frame number $t$. This is illustrated in Fig 9(a).




(a) Calculation of $\alpha j(t)$, the probability of being in state $j$ at time $t$




(b) Paths through the model with given transitions for six frames of input

Fig 9 Viterbi decoding of best state sequence

A state sequence can be thought of as a path through a state-time lattice. The term $\alpha j(t)$ is the total probability for all partial paths entering state $j$ at time $t$ as shown in Fig 9(b). The model output probability for the six frame sequence is the sum of the probability of each path through the lattice from state 1 at $t = 1$ to state 4 at $t = 6$. Note that the model cannot output a probability until a minimum of four frames has arrived. This can be seen from examining the transitions linking state 1 to state 4 on the left of Fig 9(b). It is also clear from the lattice shown in the same figure that the first path into state 4 is propagated at time $t = 4$.

It is usual to replace the calculation of $\alpha j(t)$, shown in Fig 9(a), by changing the summation operator to a 'maximum' operator. This is known as the Viterbi algorithm [3]. At each point in the lattice only the path with the highest probability is propagated forward. The model output probability thus represents the most likely state sequence rather than all possible sequences with the appropriate start and end states. The highlighted path illustrated in Fig 9(b) corresponds to a most likely sequence of $S_1S_2S_2S_3S_4S_4$.

### *5.2 State output probability — Prob($V_t$ | $j$)*

The probability of feature vector $V_t$ given state $j$ (*Prob*($V_t$ | $j$) in Fig 9) is an indicator of how closely feature vector $V_t$ matches the sound modelled by state $j$. A widespread technique is to use a continuous probability density function (pdf). Such models are known as continuous density hidden Markov models (CDHMMs). The Gaussian distribution — shown for a one-dimensional feature vector in Fig 10 — is most widely used. This models quantities which vary randomly about an expected or mean value. The popularity of the Gaussian pdf stems from the property that any pdf can be modelled by a weighted **mixture** of Gaussian pdfs. Each constituent pdf, or mode, of such a mixture can be thought of as representing a different variation (e.g. accent) of the sound within the state. A mixture with three Gaussian modes is also shown in Fig 10.

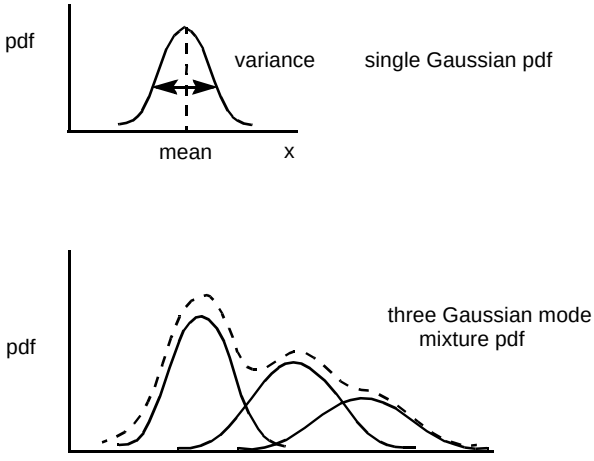



Fig 10 Probability density functions of a one-dimensional feature vector.

As feature vectors usually have more than one component, a **multivariate** Gaussian mixture pdf is used. Computation of the state output probability is greatly simplified if the feature vector components are uncorrelated. In this case each component is then described by a one-dimensional Gaussian mixture pdf and the probability of individual components are multiplied together to obtain the state output probability for the feature vector.

### 5.3 Model training

The most difficult aspect of pattern recognition, generally, is training models for pattern matching. Broadly speaking, there are two forms of training — **supervised** and **unsupervised**. In supervised training, the pre-categorised examples of the item to be recognised are input to the training process. Unsupervised training is data driven and consists of letting the training process find categories for recognition from the training data. Humans tend to be very adept at unsupervised learning, i.e. noticing a new pattern in the environment (visual, auditory) and learning it for future recognition.

Training of HMMs is normally carried out in supervised mode. As HMMs model the statistics of a signal many examples are usually required for training. These are collected in speech databases which may be stored as files of speech data. Speech data must be annotated to be of use for supervised training. Annotation consists of specifying where each speech example starts and ends, and labelling the example according to how it is to be classified. Manual annotation of speech training data usually leads to best results as the expertise of a human listener at detecting word boundaries is captured by the process.

The Baum Welch algorithm [4, 5] is used to determine model parameters such as state transition probabilities, means, and variances of Gaussian pdfs. This algorithm is iterative and requires an initial estimate of the model parameters. This initial estimate is generated during a separate phase.

### 5.4 Model initialisation

This process starts with an arbitrary assignment of feature vectors to states for each training example (e.g. word, phoneme). The vectors assigned to a given state are pooled for all training examples. When all vectors from all examples have been thus assigned, the vectors in each pool are **clustered**. This is the process of searching for $M$ clusters in $N$-dimensional feature vector space (for $M$ Gaussian modes). The mean of the vectors within a cluster becomes the initial estimate of a Gaussian mode.

The next phase of initialisation uses this estimate of the parameters, but this time the assignment of vectors to states is not arbitrary. The Viterbi algorithm, discussed earlier, is used to decide the most likely state sequence through the model for a training example, and hence to which state pool each vector is to be assigned. After the vectors from all training examples are assigned, the state pools are again clustered. The process is iterated on the training data until a suitable convergence criterion is met.

### 5.5 Baum Welch training

The Baum Welch algorithm takes an initial model $\lambda$ and estimates a new model $\lambda'$ using all the training examples. During this process the model output probability $P(V|\lambda)$ is calculated for the vectors $V$ of each training example. The algorithm guarantees that:

$$P(V|\lambda') \geq P(V|\lambda) \quad \text{..... (2)}$$

i.e. the model output probability for the training data set is at least as good as for the initial model. This algorithm is repeated using $\lambda'$ as the initial model for the next iteration.

The principle behind Baum Welch training is fairly straightforward. In statistics the mean, or expected, value of a variable $x$ can be estimated by the summation:

$$\bar{x} = \sum_{\forall x} xP(x) \quad \text{..... (3)}$$

where $P(x)$ is the probability of $x$ for all values of $x$. The current estimate of the model is used to calculate $P_j(V_t)$ the probability of being in state $j$ for feature vector $V_t$ given an entire training example. Assuming a single Gaussian pdf per state (for example) its new mean vector is the accumulated product $V_t P_j(V_t)$ over all feature vectors $V_t$ over all training examples. This is shown in Fig 11.

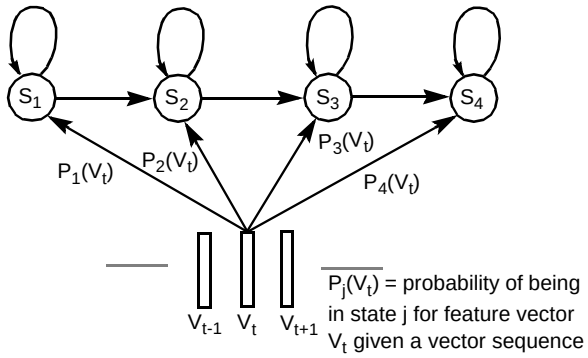



Fig 11 Model training.

For each iteration over the training data the probabilities in Fig 11 will be different from the last iteration for a given training example. The probabilities more accurately reflect the 'best' assignment of a feature vector to a state.

### 5.6 Limitations of HMMs

HMMs provide a very powerful mechanism for simultaneously modelling temporal and spectral variations of an acoustic signal. However, some assumptions are made in the application of HMMs to speech signals which

seriously affect both robustness and recognition accuracy. Much research has been done in the last decade to overcome the theoretical limitations of HMMs for modelling speech.

The most dubious assumption made in HMMs is the independence assumption. This assumes that successive feature vectors matched to a model state are **independently** and **identically** distributed. This implies that the state output probability calculation makes no allowance for the influence of preceding vectors. The state probability calculation is indifferent to improbable events such as, say, a ten-vector syllable where each vector represents speech from a different speaker. HMM variants, such as the segmental HMM [6] and inter-frame dependent HMM [7], have been postulated to overcome this limitation.

The assumption that vectors mapping to a state are identically distributed is a simplifying approximation to real speech signals. Essentially this models speech as a series of discontinuous jumps through a sequence of sounds. In reality the position of the vocal tract for one speech sound greatly influences how the following sound evolves.

Another weakness of HMMs is that sound durations are unrealistically modelled. This seems contradictory, as one of the strengths of HMMs is that they accommodate patterns which evolve through time allowing arbitrary duration signals to be matched to a model. However, the actual durations of the sounds themselves are inaccurately modelled by HMMs. Consequently, implausibly long or short model-state durations are not penalised, which affects recognition performance. Strategies to incorporate better HMM durational models are described in Levinson [8] and Russell and Moore [9].

### *5.7 Neural networks*

An alternative to statistical models such as HMMs is that of artificial neural networks (ANN). As the name suggests ANNs are inspired by theories of how neurons in the brain are organised. The input to an ANN is typically a speech feature vector. Each output may represent a recognition outcome, e.g. a sound to be recognised. An example ANN known as a multilayer perceptron is shown in Fig 12.

The inputs are the components of a feature vector. Each component connects to a node in the input layer. Each input node then connects to all the nodes in a **hidden** layer. Each output node represents a sound for the word 'fred' and is fed from all hidden nodes. The highest output node is usually taken to indicate the recognised sound.

ANNs distinguish different inputs by finding partitions between vectors for different patterns. This is the **discriminative** approach to pattern recognition. The number of nodes in the hidden layer limits the number of boundaries the perceptron can find. Adding more hidden layers allows more complicated boundary shapes.

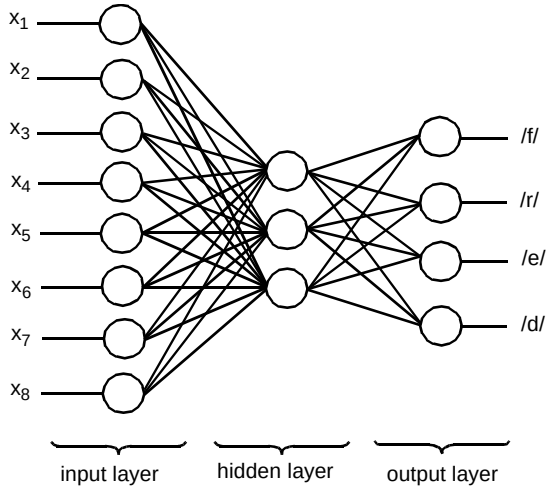



Fig 12 Multilayer perceptron for recognition.

A major drawback of ANNs is that a fixed size of input vector is required, making it difficult to deal with time varying patterns. Several ANN architectures specifically deal with this problem, e.g. time-delay neural network (TDNN) where the outputs feed back to the input layer [10].

A growing trend in pattern matching is to combine HMMs with ANNs in a hybrid system. This allows the temporal modelling capability of HMMs and the discriminative nature of ANNs to be exploited [11].

Alternatively, the standard HMM parameters can be estimated by discriminative training. This differs from Baum Welch training by optimising some function of **all** the models to be trained. Examples of such training algorithms are maximum mutual information and minimum error rate training [12]. Discriminatively trained models can be used for recognition without modification to the standard Viterbi algorithm.

## 6. Speech recognition

The previous section provided a brief overview of the 'low-level' pattern matching elements used to decode a speech waveform into a symbolic representation. This section first discusses the parser — a mechanism for searching possible model sequences to give a symbol transcription. The parser is central to tailoring speech recognisers to different applications.

Current speech recognition technology cannot transcribe spontaneous natural speech with anything like human performance — either in terms of speed or accuracy. However, with suitable constraints, recognition performance can be good enough to be usable in many situations. An example application is an automatic flight-booking system. In such a system the user can be prompted to speak an option, e.g. a destination airport or a departure time. The

spoken input can then be verified by prompting the user to say either ‘yes’ or ‘no’. Such a spoken dialogue constrains the recognition task and maximises system integrity.

### *6.1 Parsing — the recognition engine*

The previous section described how an arbitrary amount of speech can be matched to a model and a probability for that model calculated. Given a set of word models, recognition of a single spoken word could be carried out by matching the incoming speech to each model in turn, the model with the highest probability corresponding to the recognised word. More generally, the spoken input might be a word sequence. Some means of matching incoming speech feature vectors to the possible model sequences is needed to determine the most likely word sequence. This is the function of the parser.

As with state sequences within an HMM, the number of potential word sequences becomes intractable even for short utterances. An efficient parsing strategy is to extend the Viterbi algorithm to decode sequences of **models** rather than sequences of HMM states. Such a parser is described by a grammar network as illustrated in Fig 13 for a simple response to a prompt such as: ‘Please state a departure airport’.

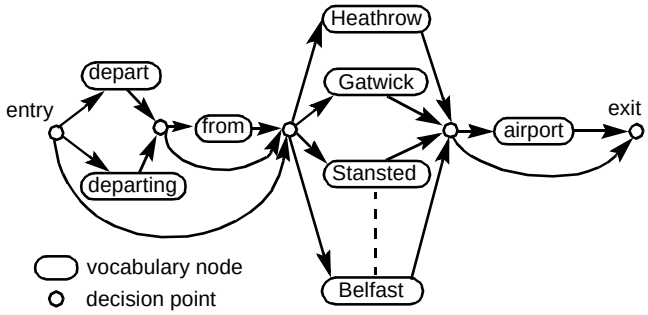



Fig 13 A simple grammar network.

The network consists of nodes which connect to other nodes. Each node represents a model of a word. The parsing algorithm assigns frames to nodes where they are matched against the associated model. As frames get propagated through the model they eventually reach the end state, when a model score becomes available. Once this happens, the parser passes path information to connecting nodes. This contains at least the score and an identifier for the previous node. The frame time may be recorded as well to provide information for other processes.

If paths from more than one predecessor node are input to a node, the path with the best score is chosen. This parsing algorithm is completely independent of any type of pattern-matching element. The more general term ‘score’, rather than ‘probability’, is used to reflect this.

### *6.2 Viterbi search efficiency*

The Viterbi search makes use of frame-synchronous processing, i.e. partial paths are evaluated at each frame. A very simple grammar to parse for ‘yes’ or ‘yes please’ is shown in Fig 14.

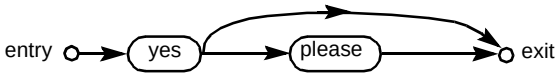



Fig 14 A ‘yes’, ‘yes please’ grammar network.

If a minimum duration of two frames for both models is assumed, the parses shown in Fig 15 are possible for a five-frame utterance.

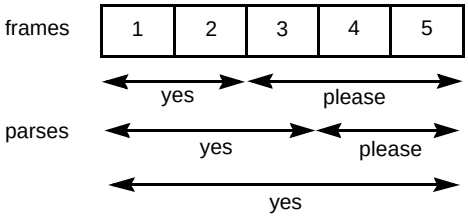



Fig 15 Possible parses of 5 frames of input.

No output appears from the ‘yes’ node until two-frames are processed after which an output path emerges from the ‘yes’ node. At the third frame the path emerging from ‘yes’ at frame two is propagated into the ‘please’ node and hence into the entry state of the ‘please’ model as shown in Fig 16.

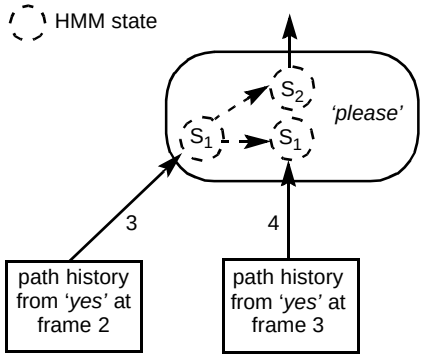



Fig 16 Intra-node decision at frame four.

At frame four, however, the same model entry state is entered, but with a new path history. This state could also have been entered by the internal state transitions of the model. Only the better scoring path into state 1 at frame four is selected and further propagated. This decides between the first two parser outputs of Fig 15 as early as frame four. An efficient implementation of this Viterbi search mechanism is described in Young et al [13].

Evaluating all partial paths at each time step affords great computational economy in searching the trajectories through the network for an utterance.

### *6.3 Isolated-word recognisers*

The simplest type of recogniser is one where a single spoken word is expected. This is known as an isolated-word recogniser. If the words are each modelled by an HMM then recognition can be performed by matching the incoming speech against each model in turn. In practice, speech does not occur in isolation — it is embedded in background noise or silence. The noise arises from channel effects as well as ambient noise such as music, dogs barking, etc.

A separate noise model is often used to match to non-speech regions of the signal. This is trained on non-speech portions of the training utterances.

A grammar network for an isolated-word recogniser is shown in Fig 17. As can be seen there is a noise node both before and after the parallel vocabulary models. The same noise model is used for both these nodes, but a different instantiation of model calculations must occur, as the parser may pass different frame sequences to these nodes for mapping to different regions of the signal.

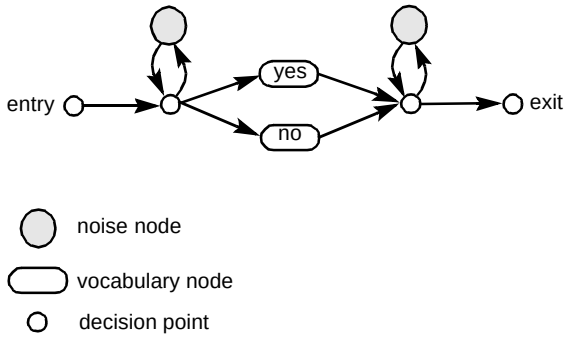



Fig 17 Grammar for an isolated word recogniser.

### *6.4 Multiple parses*

The network for the isolated-word recogniser of Fig 17 can only propagate the best path to the exit node as the previous node selects its best entry path at each time instant. For many applications that use such a recogniser, it is desirable to know what the next best recogniser hypotheses are. To do this the exit node must propagate all input paths and all desired hypotheses must feed into the exit node. The same isolated-word network is shown in Fig 18 for multiple candidate output.

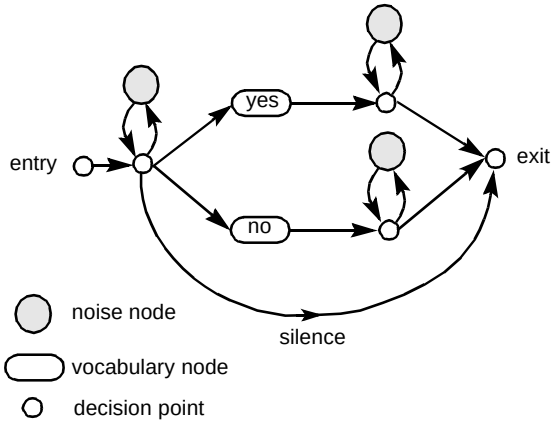



Fig 18 Isolated-digit grammar with multiple candidate output.

As can be seen from Fig 18, each vocabulary node must feed into a separate post-speech noise node so that all hypotheses are available at the output. This increases the computational cost as more noise model calculations must be done. This network also features a separate silence, or noise-only, path. This enables the recogniser to detect if nothing was spoken.

This type of isolated-word recogniser is successfully deployed in the BT CallMinder ™ service. The simplest vocabulary implemented is the ‘yes, no’ vocabulary for which the recognition accuracy is greater than 99% on live traffic.

Use of alternative recogniser hypotheses can improve overall system accuracy. For example, the top two paths can be checked for known confusable word pairs. If the top two contain a confusable pair (e.g. ‘cat’, ‘bat’) it may be appropriate to prompt the user to confirm the top choice.

### *6.5 Connected-word recognisers*

A useful component for many data retrieval systems is the ability to input a digit sequence. Recognition of connected words is considerably more difficult than isolated word recognition. This is due to co-articulation — the influence of surrounding words over the sound of a word. A simple grammar network for a connected digit recogniser is shown in Fig 19.

This is an example of an unconstrained network with post-speech noise models omitted for simplicity. Any sequence of digit nodes with optional noise nodes between digits is allowed. The recognition accuracy for an isolated digit recognition task, as mentioned earlier, is typically above 95% for speaker-independent telephony recognition using CDHMMs. If the per-digit recognition is 95% for connected digits the accuracy for ten-digit sequences is

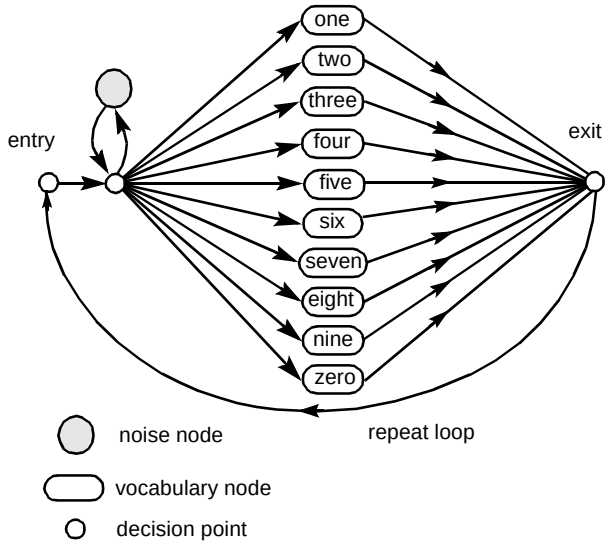



Fig 19 Unconstrained connected-digit grammar network.

$0.95^{10}$ (~ 60%). Co-articulation effects result in this figure being lower in practice. The poor durational model of HMMs, mentioned earlier, has the effect that unconstrained recognisers tend to match more words to speech than were actually spoken.

Recognition accuracy can be increased if known constraints on the input are built into the recognition process. For example, if digit strings of length three are expected then a network, such as shown in Fig 20, can be used.

If all possible digit sequences are known, e.g. for account numbers, a huge tree network with all sequences can be built. Connected-digit accuracy for such a network of say 30 000 digit strings (each eight digits in length) is currently greater than 90%.

Again, obtaining the top *N* word sequences from recognisers, such as connected digit or connected-alpha-numeric recognisers, can be used to improve overall system integrity. Unless the network is a tree network, in which case the path to every leaf node is unique, special techniques [14] are needed to provide multiple candidate output.

### *6.6 Speaker-dependent recognisers*

Speaker-dependent recognisers can be used for large vocabulary recognition, the task constraint being that they are optimised for only one user. A typical application is a dictation system. Usually a headset, with high-quality microphone (bandwidth of 0 to 20 kHz) is worn by the user. Some systems have advanced to allowing continuous speech (earlier systems requiring short pauses between words). Vocabularies of up to 60 000 words are possible. State-of-the-art dictation systems, such as those from Dragon, Kurzweil, IBM and Philips, allow the user to correct misrecognitions (on screen). This, combined with adapting a language model to the user's speech usage, enables very good recognition rates for large vocabularies.

### *6.7 Sub-word modelling*

As vocabulary size increases for both isolated- or connected-word recognisers, it becomes less feasible to collect many examples of each word. This has motivated the technique of sub-word modelling. There are a relatively small number of constituent sound elements, or phonemes, in a given language. By building phoneme models a recogniser for any vocabulary can be built once the phonetic transcription of every vocabulary word is known.

To train phoneme models a continuous speech database is required. For speaker-independent telephony recognition this typically consists of spoken sentences collected across a

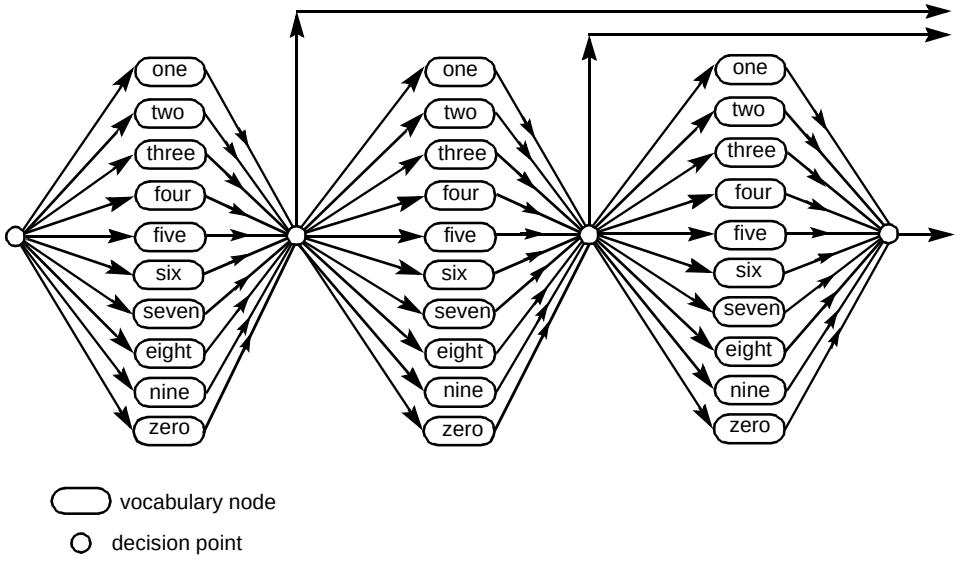



Fig 20 Grammar for strings of up to three digits.

wide range of different speakers and conditions. The sentences themselves may be devised to have good phonetic coverage of both each phoneme and the phonetic contexts in which it appears. The BT Subscriber database [15], collected across the UK over the public telephone network, is phonetically annotated. It contains five 'phonetically rich' sentences for each of 1000 talkers.

For recognition each vocabulary word is converted into a phoneme sequence using a phonetic lexicon. For words not in the lexicon, rules are applied to generate a sequence. Thus, new vocabularies can be generated 'on the fly' by what is known as rapid vocabulary generation (RVG). This offers huge advantages over the conventional approach of collecting a database for each new vocabulary and training new whole-word models. The price paid for the flexibility of RVG is that the achievable recognition accuracy is never as good as for whole-word modelling — for small, isolated-word vocabularies, recognition accuracy using RVG is typically 5% to 10% poorer. This is due to phoneme models being trained over many more phonetic contexts than specifically encountered in the target word.

More accurate modelling of a phoneme can result from considering the context in which it occurs. For example, instead of just one phone model for phoneme /a/, many bi-phone models can be created instead. A bi-phone is a model of a particular left or right context, e.g. /b/-/a/ and /c/-/a/ are both left bi-phones of phoneme /a/. For a set of 50 phonemes there are 50 left contexts and 50 right contexts that can be modelled for each phoneme. A better sub-word model is the tri-phone. Tri-phone models account for both the left **and** right context of a phoneme e.g. /c/-/a/-/t/ is a tri-phone of /a/. There are 2500 (i.e. $50^2$) possible triphones for each phoneme. Although not all of these contexts occur in practice there are still a large number of models to train. Many of the contexts will have insufficient training data even in very large databases. The focus of much sub-word modelling research is to find a suitable compromise between model specificity and trainability. Techniques for this include decision-tree clustering [16], variable-mode CDHMMs [17] and phones in context [18].

Note that bi-phones or tri-phones are still trained on the data for the phoneme represented, e.g. the /a/ sound in the tri-phone example.

## 7. Towards more natural speech input

One of the greatest challenges of speech recognition is accurate recognition of natural speech. This section discusses two approaches to relaxing constraints of speaker-independent systems. The first of these is wordspotting where natural speech is searched for keywords. The second is a large-vocabulary system which uses a language model.

### *7.1 Wordspotting*

Wordspotting is a form of search strategy on continuous speech input to detect any occurrences of a vocabulary keyword. Dialogue constraints are considerably relaxed while still extracting information from the signal. There are many applications for wordspotting. Voice control of consumer products, e.g. video recorders, televisions and PCs, can be achieved even with a small keyword vocabulary. Also, either live or recorded speech can be searched for particular topics. Even in many dialogue-controlled applications, the user's response to a prompt often contains extraneous speech such as 'ah, oh yes...' which a word-spotting recogniser is useful for filtering out.

The primary difficulty of wordspotting is accurate detection of non-keyword speech. The keyword vocabulary can be matched with specifically trained word models or with appropriate sub-word models. Generic models, trained up on a wide variety of non-keyword speech examples, are often used to match to non-keywords. These are known as sink models, shown in Fig 21. For simplicity the sink nodes in the diagram represent word models — more complex sink models can be built from either unconstrained phoneme loops or a large sub-network of phoneme sequences for common extraneous words.

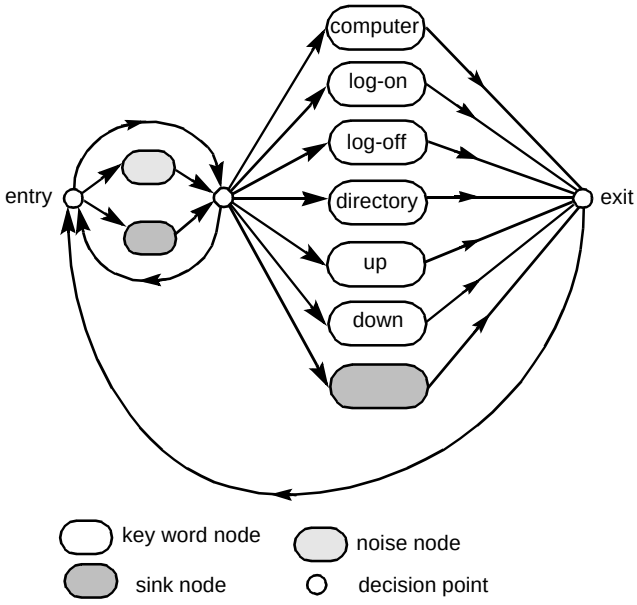



Fig 21 Wordspotting grammar network.

This grammar allows any sequence of noise or sink models, the only constraint being that keywords are assumed to be surrounded by either extraneous speech or noise.

An important feature for continuous-speech recognisers is the ability to output a hypothesised transcription as soon as it becomes available. This is known as partial traceback. To do this the parser traces back through all paths until they

converge toward a common path. This common path can then be output as a stable partial hypothesis. Even for frame-synchronous parsing such stable partial hypotheses may lag the current frame by well over half a second.

Wordspotting is particularly prone to false triggering on non-keyword speech. Information from the partial path, such as keyword or sink model scores and durations, can be processed by a rejection strategy to validate both keyword and non-keyword hypotheses and minimise the tendency to false trigger.

### *7.2 Large-vocabulary recognition*

The most exciting application of sub-word modelling techniques is undoubtedly for large-vocabulary, speaker-independent, continuous-speech recognisers. Here, a language model is used as a search constraint. The system is intended for a particular domain, e.g. medical journals. A system developed by Cambridge University Engineering Department [20] performs impressively on the US ARPA Wall Street Journal task at vocabulary sizes in excess of 20 000 words. The object of this is to recognise spoken sentences (from any speaker) taken from the Wall Street Journal.

Large-vocabulary, speaker-independent recognition requires accurate sub-word modelling — typically using thousands of tri-phone CDHMMs. In addition, a language model is applied to the parser output. This is typically a statistical model, an example of which is the *N*-gram language model which captures the probability of sequences of *N* words from the training data.

The computation and memory overhead is clearly enormous for this type of task. This can be reduced in a system with large numbers of context-dependent models by sharing model parameters both within and between models [19]. At the parsing level some form of network **pruning** must be used to improve search efficiency. With pruning, any nodes with output path scores below a threshold are deactivated and the corresponding paths removed. Careful tuning is needed to avoid the best path being pruned. Dynamic network creation, i.e. creating new nodes 'on the fly' and destroying redundant ones also renders computation more manageable. Despite this, real time performance is often unattainable even on platforms such as a Unix ™ workstation.

An in-depth overview of language models and decoding strategies for large-vocabulary continuous-speech recognition is given in Ney [21].

## 8. Results processing

The parser output results (namely word sequences with associated scores) are continually processed to detect the end of speech. Before returning the decoded speech to an application, a further processing stage known as a **rejection strategy** is often applied to validate the recognition decision. Many factors can be combined to make the rejection decision as illustrated in Fig 22.

Both the score (and score ratios) reflect the extent to which the data matched to the models. The score ratio helps to determine whether the best candidate is a clear winner or not. The rejection decision may be based on heuristic rules, found by trial and error. Alternatively, this stage can be a sophisticated pattern-classification stage in itself, using a Gaussian classifier, neural networks or fuzzy logic. The controlling application can then verify the recogniser decision by a dialogue depending on the rejection output.

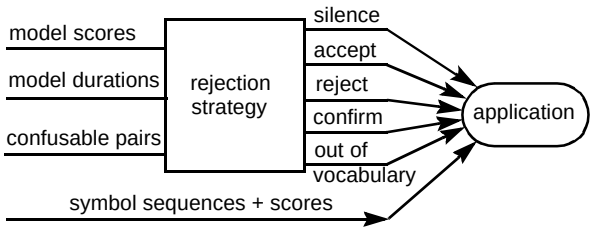



Fig 22 Rejection of recogniser decisions.

One of the most difficult conditions to detect is when the speech contains an out-of-vocabulary (OOV) word [22]. OOV speech often satisfies all the input criteria to the rejection stage, including scoring highly on an incorrect model. Detection of OOV speech is vital for robust telephony based recognisers as well as for wordspotting. The limitations of standard CDHMMs (see section 5.4) undoubtedly undermine robustness to OOV speech.

Ultimately, as large-vocabulary continuous-recogniser technology improves in both accuracy and efficiency, tasks such as wordspotting and OOV rejection will become obsolete.

## 9. Speaker recognition

Acoustic pattern-matching techniques can also be applied to recognising the speaker rather than the speech content of a signal. Speaker recognition falls into two categories — speaker **verification** and speaker **identification**. Verification is the task of validating a speaker's identity claim. Identification, on the other hand, is deciding who the speaker is. Identification systems usually assume a fixed size speaker population. Speaker-recognition systems can be text dependent where the vocabulary is pre-specified, or text independent where arbitrary speech is recognised.

- Speaker verification

  For text-dependent verification, one or more templates (or CDHMMs) are trained for each vocabulary item — usually also for each repetition.

  To verify an identity claim, the user is prompted for a vocabulary word or phrase and the input is then matched against the appropriate templates (or models) for each speaker. If the average score from the models for that speaker exceeds a threshold the user is accepted. For text-independent verification, the vector quantised (VQ) codebook (see below) for the claimant is used. The performance metric for verification systems is usually the equal error rate (EER). This is the operating point at which imposter acceptance equals valid user rejection.

- Speaker identification

  Speaker identification is very similar to verification. During recognition, speech is matched to the templates (CDHMMs or VQ codebook) for the prompted vocabulary word (text dependent) or against all templates (text independent). The speaker associated with the highest scoring template is output as the identified person.

As would be expected, speaker recognition is different in many ways to speech recognition. The problem is to extract information from the speech which is unique to an individual rather than that specific to the intelligibility of the spoken input. Interestingly, a very similar front end is often used — the main difference being that a different subset of feature vector coefficients are selected.

Statistical models such as HMMs are difficult to train for speaker recognition tasks due to the paucity of available training data. A widely used technique (the precursor to HMMs for speech recognition) is dynamic time warping (DTW). With DTW the actual frame sequence for each enrolment utterance is stored in a template. During pattern matching, incoming frames are 'warped' in time to align with each template and an accumulated frame distance used as a score [23].

CDHMMs can also be used as pattern matching elements for speaker recognition. This is an advantage if a system is to perform both speaker and speech recognition as the basic pattern matching software can be used throughout. Best performance is obtained if the number of states approximates the expected number of frames for the word. This forms a specific model of the unique sound progression for the speaker. The lack of training data makes it difficult to obtain a good estimate of the variances of the state probability distributions. For this reason a model resolution of only a single Gaussian mode per state is used. Also, only one variance — known as a grand variance — is estimated from the entire data and shared by each state's Gaussian pdf.

Using specific vocabulary templates (or models) for each speaker leads to best performance for text-dependent systems. For text-independent systems, it is futile to model the training words explicitly — instead a VQ codebook is generated for each speaker [24]. The codebook is generated by pooling all speech frames from the training data for a speaker and clustering the frames to form the desired number of clusters. Pattern matching then consists of accumulating the distance of each speech frame to the nearest cluster centre. The advantage of VQ codebooks is efficiency, but at a cost of losing temporal information of speech production. An in-depth discussion of speaker recognition is given in Furui [25].

## 10. What next?

This paper has presented an overview of the technology behind telephony-based ASR. State-of-the-art feature extraction and pattern-matching components were described first. Next, the use of parsers to guide a recognition process along suitable task constraints was explored, and finally some issues of results processing and speaker recognition were discussed.

So, where lies the future of speech and speaker recognition technology? The best speech recognition systems of today, although far short of human performance, demonstrate the power of the purely statistical acoustic and language model. On the other hand automatic speaker recognition possibly already outperforms humans. One vital direction is to improve the acoustic pattern matcher — humans learn new words with few repetitions and generalise to different speakers and conditions rather than being trained on many representative samples. There is plenty of scope for overcoming the theoretical obstacles of current techniques. Moreover, combining different classifiers — by running them in parallel and comparing their output — may improve recognition accuracy.

It seems likely that continued research into statistical techniques will lead to steady ASR performance gains. Moreover, as computing power becomes cheaper, the best systems of today will soon appear in real time applications.

In the longer term, perhaps an alternative to statistical models will bridge the gap between human and machine performance. Such models might employ techniques inspired by artificial intelligence to discover rules based on the features to characterise different sounds.

If such a change of focus occurs in the next decade real time conversations with a machine may be a commonplace occurrence not long into the next century.

## References


1 Lombard E: 'Le signe de l'elevation de la voix', Ann Maladies Oreille, Larynx, Nez, Pharynx, 37 (1911).

2 Parsons T: 'Voice and speech processing', McGraw-Hill, p 175 (1987).

3 Viterbi A J: 'Error bounds for convolutional codes and an asymptotically optimum decoding algorithm', IEEE Trans on Information Theory, IT-13 (April 1967).

4 Cox S J: 'Hidden Markov models for automatic speech recognition: theory and application', BT Technol J, 6, No 2, pp 105—115 (April 1988).

5 Huang X D, Ariki Y and Jack M A: 'Hidden Markov models for speech recognition', Edinburgh University Press (1990).

6 Holmes W and Russell M: 'Experimental evaluation of segmental HMMs', ICASSP-95 (1995).

7 Ming J and Smith F: 'Inter-frame dependent hidden Markov models for speech recognition', IEE Elect Lett, 30, pp 188—189 (1994).

8 Levinson S: 'Continuously variable duration hidden Markov models for automatic speech recognition', Computer Speech and Language, 1, No 1 (March 1986).

9 Russell M and Moore R: 'Explicit modelling of state occupancy in hidden Markov models for automatic speech recognition', ICASSP-85 (1985).

10 Robinson T, Hochberg M and Renals S: 'IPA: improved phone modelling with recurrent neural networks', ICASSP-94 (1994).

11 Morgan N and Bourland H: 'An introduction to the hybrid HMM/ connectionist approach', IEEE Signal Processing Magazine, 12, No 3 (May 1995).

12 Reichl W and Ruske G: 'Discriminative training for continuous speech recognition', EUROSPEECH-95 (1995).

13 Young S et al: 'Token passing: a simple conceptual model for connected speech recognition systems', Cambridge University Engineering Department Report (1989).

14 Ringland S: 'Applications of grammar constraints to ASR using signature functions', in: 'Speech Recognition and Coding (New Advances and Trends)', NATO ASI Series F: Computer and System Sciences, 147 (1993).

15 Simons A D and Edwards K: 'Subscriber — a phonetically annotated telephony database', Institute of Acoustics Speech and Hearing (1992).

16 Lee K F et al: 'Allophone clustering for continuous speech recognition', ICASSP-90 (1990).

17 Ollason D: 'Variable pool size tied parameter systems for context dependent sub-word unit speech recognition', Institute of Acoustics Speech and Hearing Conference (Autumn 1994).

18 Moore R et al: 'A comparison of phoneme decision tree (PDT) and context adaptive phone (CAP) based approaches to vocabulary independent speech recognition', ICASSP-94 (1994).

19 Farhat A and O'Shaughnessy D: 'A shared-distribution approach in a hidden Markov model-based continuous speech recognition system', EUROSPEECH-95 (1995).

20 Woodland P et al: 'Large-vocabulary continuous-speech recognition using HTK', ICASSP-94 (1994).

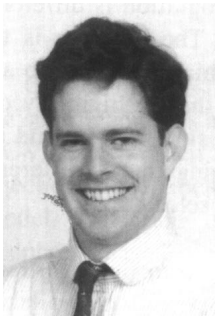
21 Ney H: 'Architecture and search strategies for large-vocabulary continuous-speech recognition', in: 'Speech Recognition and Coding (New Advances and Trends)', NATO ASI Series F: Computer and System Sciences, 147 (1993).

22 Sukkar R and Wilpon J: 'A two-pass classifier for utterance rejection in keyword spotting', ICASSP-93 (1993).

23 Furui S: 'Cepstral analysis technique for automatic speaker verification', IEEE Trans Speech and Signal Processing, ASSP-29, No 2, pp 254—272 (April 1981).

24 Matsui T and Furui S: 'A text independent speaker recognition method robust against utterance variations', ICASSP-91 (1991).

25 Furui S: 'An overview of speaker recognition technology', ESCA Workshop on Automatic Speaker Recognition, Identification and Verification, Martigny (1994).

Kevin Power graduated from University College, Cork, Ireland in 1990 with a degree in electrical and electronic engineering. He worked for Hyperion Energy Systems in Cork developing the waveform definition component of a radar control system before joining the speech technology unit at BT Laboratories in January 1991. His initial work involved downstreaming speech recognition algorithms. In 1993 Kevin joined the speech and speaker recognition research team where he developed techniques for 'out-of-vocabulary' rejection. Since then he has been investigating improved acoustic models for recognition. He is currently a project leader of the speech and speaker recognition research project.